ChargeSync

Intelligent EV Charging Reservation and Recommendation Platform

Software Requirements Specification

SE3090 – Software Engineering Frameworks | Assignment 1

BSc (Hons) in Information Technology, SLIIT — Year 3, Semester 1, 2026

Document Version 1.0

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for ChargeSync, an integrated full-stack and Agentic AI application developed for the SE3090 – Software Engineering Frameworks Assignment 1. It is intended for use by the development team, the module evaluators, and any future maintainers of the system.

1.2 Document Scope

This document covers the four core business components (Functions 1–4), their CRUD operations and functional requirements, the complete relational database schema, external interface requirements, cross-component data flow, non-functional requirements, and the Agentic AI subsystem including its human-in-the-loop approval workflow. It does not cover UI visual design specifications or low-level implementation code, which are addressed separately in the project's Architecture Decision Records (ADRs) and technical report.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
FCM	Firebase Cloud Messaging
ORM	Object-Relational Mapping
ADR	Architecture Decision Record
LLM	Large Language Model
UUID	Universally Unique Identifier
DTO	Data Transfer Object
3NF	Third Normal Form (database normalization)
RBAC	Role-Based Access Control

1.4 Document Overview

Section 2 describes the product at a high level. Section 3 details each of the four system features with their CRUD operations and requirements. Section 4 specifies the full database schema. Section 5 defines external interfaces. Section 6 describes system architecture and cross-component connections. Section 7 lists non-functional requirements. Section 8 specifies the Agentic AI subsystem. Section 9 contains supporting appendices.

2. Overall Description

2.1 Product Perspective

ChargeSync is a new, self-contained system comprising a shared ASP.NET Core Web API and PostgreSQL database, a React web portal for station owners, administrators and support staff, a Flutter mobile application for EV drivers, and an internal Python/LangGraph Agentic AI service. The Agentic AI service is never called directly by either client — all requests are routed through the ASP.NET Core backend, which also enforces authentication, authorization and business rules.

2.2 Product Functions

- Vehicle registration and AI-driven vehicle-charger compatibility checking.
- Station, charger, operating-hours and maintenance-window management for station owners.
- Conflict-free reservation booking with waitlist handling, QR check-in, and AI-generated charging plans.
- Charging session tracking, automated invoicing, membership subscriptions, loyalty rewards and support ticket triage.
- A four-agent Agentic AI subsystem with a mandatory human-in-the-loop approval gate on high-impact actions.

2.3 User Classes and Characteristics

Role	Description
EV Driver	Primary mobile-app user. Registers vehicles, discovers compatible stations, reserves charging slots, tracks sessions, manages membership, loyalty and support tickets.
Station Owner	Manages their own stations and chargers via the React portal; monitors bookings, sets pricing/hours, and reviews utilization analytics.
Platform Administrator	Has full administrative access; approves station registrations, manages users, and reviews high-impact AI-flagged actions.
Customer Support Manager	Handles support tickets, session and payment disputes, and coordinates with station owners on driver issues.

2.4 Operating Environment

- Backend: ASP.NET Core Web API (C#), deployed on a cloud application host.
- Database: PostgreSQL, deployed with restricted credentials and applied migrations.
- Web Client: React, served as a static/SPA build accessible via any modern browser.
- Mobile Client: Flutter, targeting Android (APK) as the primary distribution format.
- Agentic AI Service: Python + FastAPI + LangGraph, containerized and reachable only by the backend.

2.5 Design and Implementation Constraints

- React and Flutter must communicate only with the shared ASP.NET Core Web API.
- The Agentic AI service must be an internal service called by ASP.NET Core; it must never be called directly by either client.
- No external payment gateway is used; payments are tracked via an internal invoice ledger (see ADR-05).
- No IoT/hardware telemetry is used; charging session energy is computed mathematically from rated charger output and duration (see ADR-04).
- All services must be deployable using free-tier or institution-provided cloud resources.

2.6 Assumptions and Dependencies

- Drivers and station owners have access to a smartphone/browser and a stable internet connection.
- Google Maps API and Firebase Cloud Messaging free tiers remain available and within rate limits during development and evaluation.
- The selected LLM provider (Ollama locally, or a hosted free-tier API when deployed) is reachable by the Agentic AI service at runtime.

3. System Features

3.1 Function 1 — Vehicle & AI Compatibility Discovery

Owning Student: Student 1

Allows drivers to register and manage their vehicles, discover nearby stations, and receive an AI-computed compatibility score before travelling to a charger — preventing wasted trips to incompatible or unsuitable chargers.

CRUD Operations

Operation	Description
Create	Register a new vehicle (make, model, connector type, battery capacity, max charge rate).
Read	View own vehicle list and details; search nearby stations; view compatibility results.
Update	Edit vehicle details (e.g., updated license plate, corrected specifications).
Delete	Remove a vehicle no longer in use.

Functional Requirements

Req. ID	Requirement Description	Priority
FR-1.1	Driver shall be able to register a new vehicle with make, model, connector type, battery capacity and max charge rate.	High
FR-1.2	Driver shall be able to view, update and delete their own registered vehicles only.	High
FR-1.3	System shall compute a compatibility score between a vehicle and a selected charger, including estimated charge time.	High
FR-1.4	System shall suggest alternative compatible stations when incompatibility is detected.	Medium
FR-1.5	Driver shall be able to search nearby stations filtered by connector type and distance.	High

API Endpoints

Method	Endpoint	Purpose
POST	/api/vehicles	Register a new vehicle
GET	/api/vehicles	List/filter the current driver's vehicles
GET	/api/vehicles/{id}	Retrieve a single vehicle's details
PUT	/api/vehicles/{id}	Update vehicle details
DELETE	/api/vehicles/{id}	Remove a vehicle
GET	/api/stations/nearby	Search nearby stations by location and connector type
GET	/api/stations/{id}/compatibility	Get compatibility score for a vehicle/station pair

3.2 Function 2 — Station, Charger & Operating-Hours Management

Owning Student: Student 2

Allows station owners to register and manage stations and chargers, define operating hours and maintenance windows, and reconciles all three into a real-time availability calendar with utilization analytics.

CRUD Operations

Operation	Description
Create	Register a new station; add a new charger to a station; schedule a maintenance window.
Read	View own stations, chargers, operating hours, current availability, and utilization analytics.
Update	Edit station details, charger specifications, operating hours, and charger status.
Delete	Remove a charger or station (subject to no active reservations).

Functional Requirements

Req. ID	Requirement Description	Priority
FR-2.1	Station Owner shall be able to register a new station with location and descriptive details.	High
FR-2.2	Station Owner shall be able to add, update and remove chargers under their own station(s) only.	High
FR-2.3	Station Owner shall be able to configure operating hours per day of week.	High
FR-2.4	Station Owner shall be able to schedule a maintenance window for a specific charger.	Medium
FR-2.5	System shall compute real-time charger availability by reconciling operating hours, maintenance windows and active reservations.	High
FR-2.6	System shall provide utilization and revenue analytics per station.	Medium

API Endpoints

Method	Endpoint	Purpose
POST	/api/stations	Register a new station
GET	/api/stations	List/filter/search stations
GET	/api/stations/{id}	Retrieve station details
PUT	/api/stations/{id}	Update station details
DELETE	/api/stations/{id}	Remove a station
POST	/api/stations/{id}/chargers	Add a charger to a station
PUT	/api/chargers/{id}	Update charger specifications
PUT	/api/chargers/{id}/status	Update charger status (Available/Unavailable/Maintenance)
GET	/api/stations/{id}/utilization	Retrieve utilization and revenue analytics

3.3 Function 3 — Reservation & AI Charging Planning

Owning Student: Student 3

The core Agentic AI showcase — drivers submit a charging objective (deadline, distance, price preference), and the Planning Agent coordinates with Functions 1 and 2 to produce a ranked, structured charging itinerary. Also owns conflict-free reservation locking, waitlist management, and QR check-in.

CRUD Operations

Operation	Description
Create	Create a reservation; generate an AI charging plan; add a waitlist entry.
Read	View own reservations, plan options, waitlist position, and full status history.
Update	Modify an existing reservation's time slot (subject to availability).
Delete	Cancel a reservation, triggering automatic waitlist promotion.

Functional Requirements

Req. ID	Requirement Description	Priority
FR-3.1	Driver shall be able to create a reservation for an available charger slot.	High
FR-3.2	System shall prevent double-booking using database-level exclusion constraints and row-level locking.	High
FR-3.3	Driver shall be able to cancel or modify an existing reservation.	High
FR-3.4	System shall maintain a waitlist per charger and automatically promote the next entry on cancellation.	Medium
FR-3.5	Driver shall be able to check in to a reservation via QR code scan.	High
FR-3.6	System shall generate an AI-ranked charging plan given a driver's objective (deadline, distance, price preference).	High
FR-3.7	System shall record a full status history for every reservation.	Medium

API Endpoints

Method	Endpoint	Purpose
POST	/api/reservations	Create a reservation
GET	/api/reservations	List/filter/sort/paginate reservations
GET	/api/reservations/{id}	Retrieve reservation details
PUT	/api/reservations/{id}/cancel	Cancel a reservation
POST	/api/reservations/{id}/checkin	QR check-in to start a session
POST	/api/charging-plan/generate	Generate an AI-ranked charging plan
GET	/api/reservations/{id}/history	Retrieve reservation status history

3.4 Function 4 — Session, Payment, Loyalty & Support Management

Owning Student: Student 4

Manages the full post-booking driver relationship: software-based charging-session tracking (no IoT hardware required), automatic invoice generation, membership subscriptions, loyalty points/tiers, reward redemption, and support ticket triage — with high-impact refund/redemption actions gated behind human approval.

CRUD Operations

Operation	Description
Create	Start a charging session; generate an invoice; create a subscription; submit a support ticket; redeem a reward.
Read	View session history, invoices, subscription status, loyalty balance/tier, and support ticket status.
Update	Stop/complete a session; change a subscription plan; update a support ticket's status.
Delete	Cancel a subscription; withdraw a pending support ticket.

Functional Requirements

Req. ID	Requirement Description	Priority
FR-4.1	System shall record session start/stop and compute energy delivered as Charger Output (kW) × Duration (Hours).	High
FR-4.2	System shall automatically generate a payment invoice upon session completion.	High
FR-4.3	Driver shall be able to view payment and invoice history.	Medium
FR-4.4	Driver shall be able to subscribe to a membership plan and request an upgrade/downgrade.	High
FR-4.5	System shall track loyalty points and automatically calculate membership tier.	Medium
FR-4.6	Driver shall be able to redeem loyalty points for rewards, subject to validation rules.	High
FR-4.7	Driver shall be able to submit and track support tickets.	High
FR-4.8	System shall flag sessions with duration exceeding 150% of the estimated time as anomalies.	Medium
FR-4.9	System shall flag refund requests and high-value redemptions for mandatory human approval.	High

API Endpoints

Method	Endpoint	Purpose
POST	/api/sessions/start	Start a charging session
PUT	/api/sessions/{id}/stop	Stop/complete a charging session
GET	/api/payments/invoices/{userId}	Retrieve a driver's invoice history
POST	/api/subscriptions	Subscribe to a membership plan

Method	Endpoint	Purpose
PUT	/api/subscriptions/{id}/change	Request a plan upgrade/downgrade
GET	/api/loyalty/{userId}	Retrieve loyalty balance and tier
POST	/api/loyalty/redeem	Redeem loyalty points for a reward
POST	/api/support-tickets	Submit a support ticket
PUT	/api/support-tickets/{id}/status	Update a support ticket's status

4. Data Requirements (Database Design)

4.1 Entity Relationship Overview

The database is normalized to Third Normal Form (3NF). Key relationships are summarized below; the full ER diagram is maintained separately in database/er-diagram.png.

- Users (1) — (N) Vehicles, Stations (as Owner), Reservations, Subscriptions, SupportTickets
- Vehicles (N) — (1) ConnectorTypes; Vehicles (1) — (1) VehicleChargingProfiles
- Stations (1) — (N) Chargers, OperatingHours
- Chargers (1) — (N) MaintenanceWindows, Reservations, WaitlistEntries
- Reservations (1) — (N) ReservationStatusHistory; Reservations (1) — (1) ChargingSessions
- ChargingSessions (1) — (1) PaymentInvoices
- Users (1) — (1) LoyaltyAccounts; LoyaltyAccounts (1) — (N) RewardRedemptions
- AgentWorkflowRuns references Users (Driver and Approver) and is referenced by ChargingPlans

4.2 Data Dictionary

Users (shared)

Stores all system accounts across all four roles.

Column	Data Type	Constraints	Description
Id	UUID	PK, DEFAULT gen_random_uuid()	Unique user identifier
FullName	VARCHAR(150)	NOT NULL	User's full name
Email	VARCHAR(150)	NOT NULL, UNIQUE	Login email address
PasswordHash	VARCHAR(255)	NOT NULL	Bcrypt-hashed password
Role	VARCHAR(30)	NOT NULL, CHECK IN (Driver, StationOwner, Admin, SupportManager)	Assigned system role
PhoneNumber	VARCHAR(20)	NULL	Contact phone number
IsActive	BOOLEAN	NOT NULL, DEFAULT TRUE	Account active/suspended flag
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

ConnectorTypes (Function 1)

Lookup table of supported EV connector standards (e.g., CCS2, CHAdeMO, Type2).

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique connector type identifier
Name	VARCHAR(50)	NOT NULL, UNIQUE	Connector standard name
Description	TEXT	NULL	Additional details
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

Vehicles (Function 1)

Driver-owned vehicle records used for compatibility scoring.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique vehicle identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Owning driver
Make	VARCHAR(50)	NOT NULL	Vehicle manufacturer
Model	VARCHAR(50)	NOT NULL	Vehicle model
Year	SMALLINT	NULL	Manufacture year
ConnectorTypeId	UUID	NOT NULL, FK -> ConnectorTypes(Id)	Vehicle's connector standard
BatteryCapacityKwh	DECIMAL(6,2)	NOT NULL	Total battery capacity
MaxChargeRateKw	DECIMAL(6,2)	NOT NULL	Maximum charge rate supported
LicensePlate	VARCHAR(20)	NULL	Vehicle registration plate
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

VehicleChargingProfiles (Function 1)

Extended charging preferences/profile data used by the Compatibility Agent.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique profile identifier
VehicleId	UUID	NOT NULL, UNIQUE, FK -> Vehicles(Id)	Associated vehicle
OptimalChargeRateKw	DECIMAL(6,2)	NULL	Manufacturer-recommended optimal rate

Column	Data Type	Constraints	Description
PreferredConnectorTypeId	UUID	NULL, FK -> ConnectorTypes(Id)	Preferred connector if multiple supported
NotesJson	JSONB	NULL	Additional structured profile data
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

Stations (Function 2)

Charging stations registered by station owners.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique station identifier
OwnerId	UUID	NOT NULL, FK -> Users(Id)	Owning station owner
Name	VARCHAR(150)	NOT NULL	Station display name
AddressLine	VARCHAR(255)	NOT NULL	Physical address
Latitude	DECIMAL(9,6)	NOT NULL	Geolocation latitude
Longitude	DECIMAL(9,6)	NOT NULL	Geolocation longitude
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN (Pending, Active, Suspended)	Approval/operational status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

Chargers (Function 2)

Individual charging units belonging to a station.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique charger identifier
StationId	UUID	NOT NULL, FK -> Stations(Id)	Parent station
ConnectorTypeId	UUID	NOT NULL, FK -> ConnectorTypes(Id)	Supported connector standard
MaxOutputKw	DECIMAL(6,2)	NOT NULL	Rated maximum power output
PricePerKwh	DECIMAL(6,2)	NOT NULL	Price charged per kWh delivered

Column	Data Type	Constraints	Description
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Available', CHECK IN (Available, Unavailable, Maintenance)	Current operational status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

OperatingHours (Function 2)

Weekly operating schedule per station.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique record identifier
StationId	UUID	NOT NULL, FK -> Stations(Id)	Associated station
DayOfWeek	SMALLINT	NOT NULL, CHECK (0-6)	0 = Sunday ... 6 = Saturday
OpenTime	TIME	NOT NULL	Daily opening time
CloseTime	TIME	NOT NULL	Daily closing time
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

MaintenanceWindows (Function 2)

Scheduled downtime periods for a specific charger.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique record identifier
ChargerId	UUID	NOT NULL, FK -> Chargers(Id)	Associated charger
StartTime	TIMESTAMPTZ	NOT NULL	Maintenance start
EndTime	TIMESTAMPTZ	NOT NULL	Maintenance end
Reason	TEXT	NULL	Maintenance description
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

Reservations (Function 3)

Driver bookings for a charger slot. Protected against double-booking via a PostgreSQL exclusion constraint on (ChargerId, tsrange(StartTime, EndTime)).

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique reservation identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Booking driver
ChargerId	UUID	NOT NULL, FK -> Chargers(Id)	Reserved charger
VehicleId	UUID	NOT NULL, FK -> Vehicles(Id)	Vehicle to be charged
StartTime	TIMESTAMPTZ	NOT NULL	Reservation start
EndTime	TIMESTAMPTZ	NOT NULL	Reservation end
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN (Pending, Confirmed, Cancelled, Completed)	Current reservation status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

ReservationStatusHistory (Function 3)

Full audit trail of every reservation status change.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique record identifier
ReservationId	UUID	NOT NULL, FK -> Reservations(Id)	Associated reservation
OldStatus	VARCHAR(20)	NULL	Status before the change
NewStatus	VARCHAR(20)	NOT NULL	Status after the change
ChangedByUserId	UUID	NULL, FK -> Users(Id)	User who triggered the change
ChangedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Timestamp of the change

WaitlistEntries (Function 3)

Queue of drivers waiting for a fully booked charger slot.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique entry identifier

Column	Data Type	Constraints	Description
ChargerId	UUID	NOT NULL, FK -> Chargers(Id)	Desired charger
DriverId	UUID	NOT NULL, FK -> Users(Id)	Waiting driver
RequestedStartTime	TIMESTAMPTZ	NOT NULL	Preferred start time
Priority	INT	NOT NULL, DEFAULT 0	Queue priority ranking
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Waiting', CHECK IN (Waiting, Promoted, Expired)	Current waitlist status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

ChargingPlans (Function 3)

AI-generated itinerary options produced by the Planning Agent for a given driver objective.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique plan identifier
WorkflowRunId	UUID	NULL, FK -> AgentWorkflowRuns(WorkflowRunId)	Originating agent workflow run
DriverId	UUID	NOT NULL, FK -> Users(Id)	Requesting driver
Deadline	TIMESTAMPTZ	NOT NULL	Driver's charging deadline
MaxDistanceKm	DECIMAL(6,2)	NULL	Driver's maximum acceptable distance
PricePreference	VARCHAR(20)	NULL	Driver's price sensitivity preference
PlanJson	JSONB	NOT NULL	Structured multi-step plan and ranked options
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

ChargingSessions (Function 4)

Software-tracked charging session lifecycle. EnergyDeliveredKwh is computed as Charger Output (kW) x Duration (Hours) — no IoT hardware required.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique session identifier
ReservationId	UUID	NOT NULL, UNIQUE, FK -> Reservations(Id)	Associated reservation
StartTime	TIMESTAMPTZ	NOT NULL	Session start (on QR check-in)
EndTime	TIMESTAMPTZ	NULL	Session end (on QR check-out)
EnergyDeliveredKwh	DECIMAL(8,2)	NULL	Computed energy delivered
Status	VARCHAR(20)	NOT NULL, DEFAULT 'InProgress', CHECK IN (InProgress, Completed, Anomaly)	Current session status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

PaymentInvoices (Function 4)

Internal invoice ledger record, auto-generated on session completion (no external payment gateway).

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique invoice identifier
SessionId	UUID	NOT NULL, UNIQUE, FK -> ChargingSessions(Id)	Associated session
DriverId	UUID	NOT NULL, FK -> Users(Id)	Billed driver
Amount	DECIMAL(10,2)	NOT NULL	Total invoice amount
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN (Pending, Paid, Refunded)	Current invoice status
IssuedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Invoice issue timestamp
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

MembershipPlans (Function 4)

Catalog of available subscription tiers.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique plan identifier
Name	VARCHAR(50)	NOT NULL	Plan display name
Description	TEXT	NULL	Plan details
MonthlyFee	DECIMAL(8,2)	NOT NULL	Monthly subscription fee
DiscountPercentage	DECIMAL(5,2)	NOT NULL, DEFAULT 0	Per-session discount for subscribers
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

Subscriptions (Function 4)

A driver's active or historical membership plan enrollment.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique subscription identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Subscribing driver
PlanId	UUID	NOT NULL, FK -> MembershipPlans(Id)	Selected plan
StartDate	DATE	NOT NULL	Subscription start date
EndDate	DATE	NULL	Subscription end date
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Active', CHECK IN (Active, Cancelled, Expired)	Current subscription status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

LoyaltyAccounts (Function 4)

One-per-driver loyalty points balance and tier.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique account identifier
DriverId	UUID	NOT NULL, UNIQUE, FK -> Users(Id)	Associated driver
PointsBalance	INT	NOT NULL, DEFAULT 0	Current points balance

Column	Data Type	Constraints	Description
Tier	VARCHAR(20)	NOT NULL, DEFAULT 'Bronze', CHECK IN (Bronze, Silver, Gold)	Current loyalty tier
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

RewardRedemptions (Function 4)

Records of points redeemed for a reward; high-value redemptions require Admin approval.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique redemption identifier
LoyaltyAccountId	UUID	NOT NULL, FK -> LoyaltyAccounts(Id)	Redeeming account
PointsRedeemed	INT	NOT NULL	Points spent on this redemption
RewardDescription	VARCHAR(255)	NOT NULL	Description of the reward
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK IN (Pending, Approved, Rejected)	Redemption approval status
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

SupportTickets (Function 4)

Driver-submitted support requests, triaged by the Validation & Support Agent.

Column	Data Type	Constraints	Description
Id	UUID	PK	Unique ticket identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Submitting driver
Category	VARCHAR(50)	NOT NULL	Ticket category (e.g., Session, Payment, Station)
Priority	VARCHAR(20)	NOT NULL, DEFAULT 'Medium', CHECK IN (Low, Medium, High, Urgent)	AI-triaged urgency
Description	TEXT	NOT NULL	Driver's issue description

Column	Data Type	Constraints	Description
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Open', CHECK IN (Open, InProgress, Resolved, Closed)	Current ticket status
AssignedToUserId	UUID	NULL, FK -> Users(Id)	Assigned support staff member
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last update timestamp

AgentWorkflowRuns (shared, Agentic AI)

Central audit table persisting the full state of every Agentic AI workflow execution.

Column	Data Type	Constraints	Description
WorkflowRunId	UUID	PK, DEFAULT gen_random_uuid()	Unique workflow run identifier
DriverId	UUID	NOT NULL, FK -> Users(Id)	Requesting driver
Objective	TEXT	NOT NULL	Driver's stated objective
Status	VARCHAR(32)	NOT NULL	Running, PendingApproval, Completed, or Failed
PlanJson	JSONB	NULL	Structured multi-step plan
AgentOutputsJson	JSONB	NULL	Each agent's individual output
ValidationResultJson	JSONB	NULL	Validation & Support Agent's result
RequiresApproval	BOOLEAN	NOT NULL, DEFAULT FALSE	Whether human approval is required
ApprovalDecision	VARCHAR(16)	NULL	Approved, Rejected, or Revised
ApprovedByUserId	UUID	NULL, FK -> Users(Id)	Approving administrator/owner/support staff
ApprovalNotes	TEXT	NULL	Reviewer's notes
FailureReason	TEXT	NULL	Safe-failure explanation, if applicable
RetryCount	INT	NOT NULL, DEFAULT 0	Number of retry attempts
CreatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT clock_timestamp()	Record creation timestamp
UpdatedAt	TIMESTAMPTZ	NOT NULL, DEFAULT clock_timestamp()	Last update timestamp

5. External Interface Requirements

5.1 User Interfaces

The React web portal provides role-based navigation for Station Owners, Platform Administrators and Customer Support Managers, including dashboards, CRUD forms, approval queues and analytics views with loading, empty, success and error states. The Flutter mobile application provides driver-facing screens for vehicle registration, station search, reservation booking, QR check-in, session/invoice history, membership/loyalty management and support ticket submission.

5.2 API Interfaces

All client-server communication uses RESTful JSON over HTTPS, secured with JWT bearer authentication and role-based authorization. Full endpoint documentation is auto-generated via Swagger/OpenAPI at the API's /swagger endpoint.

5.3 Software Interfaces

- PostgreSQL: accessed via Entity Framework Core with the Npgsql provider.
- Agentic AI Service: a FastAPI application exposed internally, called only by the ASP.NET Core AgentClient module.

5.4 Communication Interfaces

All external communication occurs over HTTPS. Internal communication between ASP.NET Core and the Agentic AI service occurs over an internal HTTP/REST connection not exposed to either client application.

5.5 Third-Party Interfaces

- Google Maps API (Places & Distance Matrix): geospatial station discovery, address geocoding, and driving-time calculation, used by Function 1 and the Planning Agent.
- Firebase Cloud Messaging (FCM): push notifications to the Flutter app for reservation confirmations, pending approvals, and support ticket updates.

6. System Architecture and Component Interconnections

6.1 High-Level Architecture

React and Flutter clients communicate exclusively with the ASP.NET Core Web API over HTTPS/REST/JSON. The API layer handles authentication, authorization, business rules, persistence via PostgreSQL, and initiates Agentic AI workflows through the internal AgentClient module. The Agentic AI service (Python/FastAPI/LangGraph) executes the four agents, persists shared workflow state to the AgentWorkflowRuns table via the API, and pauses for human approval when a high-impact action is detected. Third-party services (Google Maps, FCM) are called by the ASP.NET Core backend on behalf of the clients.

6.2 Cross-Function Data Flow

From	To	Data Exchanged	Trigger
Function 1 (Vehicle)	Function 3 (Planning Agent)	Vehicle profile and compatibility data	Charging-plan generation request
Function 2 (Station)	Function 3 (Planning Agent)	Station availability, pricing and utilization score	Charging-plan generation request
Function 3 (Reservation)	Function 4 (Session)	Confirmed reservation record	Driver QR check-in
Function 4 (Session)	Function 4 (Invoice)	Completed session energy/duration data	Session stop event
Function 4 (Loyalty)	Function 4 (Invoice)	Loyalty tier discount rate	Invoice generation
AgentWorkflowRun (shared)	All Functions	Workflow state, plan JSON, validation result, approval decision	Any agentic workflow execution
React Admin/Owner Portal	AgentWorkflowRun	Approval / rejection / revision decision	High-impact action pending
Flutter Driver App	ASP.NET Core API	All CRUD requests, JWT auth token	Every user-initiated action

6.3 Required Cross-Platform Workflow

- 1. Flutter (Driver): submits a charging objective (deadline, distance, price preference).
- 2. ASP.NET Core: authenticates and validates the request, creates an AgentWorkflowRun record.
- 3. Agentic AI: the Planning Agent coordinates with the Compatibility and Station Analysis Agents, and the Validation & Support Agent checks the result against business rules.
- 4. If a high-impact action is detected, the workflow pauses in PendingApproval status.
- 5. React (Owner/Admin/Support): reviews the proposal and approves, rejects, or requests revision.
- 6. Shared Status Update: the backend commits the approved action; the driver's Flutter app and the relevant dashboard both reflect the updated status, with an FCM push notification sent to the driver.

7. Non-Functional Requirements

7.1 Performance

- Standard CRUD API endpoints shall respond within 500ms under normal load (95th percentile).
- Agentic AI workflow execution (excluding time spent awaiting human approval) shall complete within 10 seconds under normal conditions.

7.2 Security

- All endpoints requiring authentication shall be protected via JWT bearer tokens.
- Role-based authorization shall restrict access strictly to a user's own data, except for Platform Administrators.
- Passwords shall be stored using a secure hashing algorithm (e.g., bcrypt); never in plain text.
- All API keys and secrets shall be stored in environment variables, never committed to source control.
- All database queries shall use parameterized queries via Entity Framework Core to prevent SQL injection.

7.3 Reliability and Availability

- Multi-step operations (e.g., session completion, invoice generation, loyalty point grant) shall execute within isolated database transactions.
- The system shall handle third-party service failures (Maps API, FCM) gracefully, without blocking core reservation functionality.
- Agentic AI workflows shall apply timeouts and retry limits, and produce a safe, auditable failure outcome if a step cannot complete.

7.4 Usability

- All UI views shall provide clear loading, empty, success and error states.
- Navigation shall be role-based, showing only functions relevant to the logged-in user's role.

7.5 Maintainability and Scalability

- The backend shall follow a layered architecture (Api / Application / Domain / Infrastructure) to separate concerns and support unit testing.
- The Agentic AI service shall be independently deployable and horizontally scalable, decoupled from the main API process.

8. Agentic AI Subsystem Requirements

8.1 Agent Overview

Agent	Responsibility	Input Contract	Output Contract
Vehicle Compatibility Agent	Validates hardware compatibility between a vehicle profile and charger specs; suggests alternatives if incompatible.	{vehicleId, connectorType, batteryCapacityKwh, maxChargeRateKw, stationId}	{compatible, estimatedChargeTimeMin, warnings[], suggestedAlternativeStationIds[]}
Station Analysis Agent	Evaluates live charger status, pricing history and utilization to supply candidate station scores.	{stationId, timeWindow, requestContext}	{availabilityScore, priceCompetitiveness, utilizationInsight, flags[]}
Charging Recommendation & Planning Agent (Coordinator)	Receives the driver's objective, delegates to Agents 1 and 2, ranks candidate options, builds the plan.	{driverId, vehicleId, deadline, maxDistanceKm, pricePreference}	{plan: Step[], rankedOptions[]}
Validation & Support Agent	Validates session/payment records, inspects redemptions, triages support tickets, flags high-impact actions for approval.	{taskType, userId, contextData}	{isValid, requiresApproval, summary, violations[]}

8.2 Human-in-the-Loop Approval Triggers

To prevent autonomous AI agents from executing unapproved monetary or operational actions, the following outputs trigger a PendingApproval state that halts execution until human confirmation is granted via the React web dashboard.

Trigger Condition	Threshold / Rule	Assigned Reviewer
Wallet / invoice refund request	AI-triaged support ticket recommends a refund exceeding $15.00	Customer Support Manager

Trigger Condition	Threshold / Rule	Assigned Reviewer
High-threshold loyalty redemption	Redemption exceeds 5,000 points or $50.00 equivalent value	Platform Administrator
Waitlist reservation override	AI-generated itinerary bumps a lower-priority waitlisted booking	Station Owner

8.3 Minimum Acceptance Workflow

At least one complete assessed workflow shall: (1) receive a driver's domain objective; (2) create a structured multi-step plan; (3) delegate steps to the Compatibility, Station Analysis and Planning agents; (4) call only allow-listed tools with validated inputs and structured outputs; (5) persist workflow state in AgentWorkflowRuns; (6) apply deterministic validation via the Validation & Support Agent; (7) pause for human approval on any high-impact action; and (8) produce either an auditable confirmed result or a safe, clearly recorded failure.

9. Appendices

9.1 Summary of Architecture Decision Records

ADR	Title	Summary
ADR-01	State Management Frameworks	Riverpod (Flutter) and TanStack Query / React Query (React) selected for predictable, testable client-side state.
ADR-02	Agentic AI Framework & Orchestration	LangGraph selected for structured, graph-based multi-agent orchestration with deterministic state transitions.
ADR-03	Concurrency Control for Slot Allocation	PostgreSQL exclusion constraints combined with pessimistic row-locking (SELECT ... FOR UPDATE) chosen over optimistic concurrency to guarantee zero double-bookings.
ADR-04	Software-Based Session Simulation	Session duration/energy tracked via time-based calculation rather than IoT telemetry, removing hardware dependency while satisfying full session/invoice requirements.
ADR-05	Internal Payment Invoice Ledger	An internal transaction/invoice ledger replaces a real payment gateway, avoiding compliance overhead and third-party downtime risk.

9.2 Revision History

Version	Date	Description
1.0	17 August 2026	Initial SRS covering all four functions, full data dictionary, interface requirements, architecture, non-functional requirements and Agentic AI subsystem.